AI-USE DECLARATION

CpE 211 — Laboratory 2: Linear Search in Real Applications (LibraryFinder)

Student Name	Kane Paul A. Mari
Course / Lab	CpE 211 — Laboratory 2 (Linear Search)
Instructor / Section	[Instructor Name] / [Section]
Date	October 23, 2026
Files Covered	`LibraryFinder.java`, manual trace tables, `AI-Use-Declaration.pdf` (this document)

1. Instructor-Approved Tool and Model

Item	Declared Value
AI tool	OpenCode CLI (terminal coding assistant)
Model	GLM 5.3 Flash — model ID `opencode/glm-5.3-flash-free`
Approval	Permitted by the instructor's AI-use policy for laboratory work with mandatory disclosure (this declaration).

2. Prompts Used (chronological, lightly edited for spelling)

#	Prompt	Purpose
1	*"Act as a Java tutor. Help me design pseudocode for Linear Search in Real Applications. Do not write full code until I explain the algorithm back to you."*	Constrain the AI to teach pseudocode only, no implementation.
2	*"So a linear search is basically going through each item from start to end until you find a match or run out of items. I do not know why we return -1 and not 0. Best case is the target is at the start … Build LibraryFinder with findFirst(String[] titles, String target) and countMatches(String[] titles, String target). Print every inspected index."*	Explain-back check while asking for corrections; stated the required method signatures and trace printing.
3	*"Manual trace: Trace searches for 'Java' and 'Python' in [C, Java, DSA, Java, SQL]. Count every comparison. Target at first position. Target at last position. Target missing. Repeated titles including adjacent duplicates."*	Locked the exact test arrays, the four required trace cases, and comparison counting.
4	*"You may write now. After writing, tell me how to run the code."*	Approved implementation after refining requirements.
5	*"Make an AI-use declaration in pdf. AI-use declaration: instructor-approved model, prompts used, AI suggestion adopted/rejected, and my verification."*	Requested this document.

3. AI Suggestions — Adopted

1. **Pseudocode-first workflow:** design presented as pseudocode, implementation allowed only after the explain-back step.
2. **Sentinel `-1` for "not found"** in `findFirst` (0 excluded because it is a valid array index), and `0` matches for `countMatches`.
3. **Early-return design:** `findFirst` returns on first match (best case O(1)) while `countMatches` performs a full scan (always Θ(n)) because a later match would otherwise be missed.
4. `equalsIgnoreCase()` for case-insensitive title matching, with null-argument guards.
5. **Embedded comparison counter:** each search prints `Comparisons made: N` so the program output doubles as verifiable test evidence.
6. **Two test arrays:** `[C, Java, DSA, Java, SQL]` (first / last / missing / repeated cases) and `[Java, Java, DSA, SQL]` (adjacent duplicates).

4. AI Suggestions — Rejected or Modified

AI Suggestion	My Decision
Exact, case-sensitive matching (String.equals) was recommended as the simpler lab default.	**Modified:** chose case-insensitive matching (`equalsIgnoreCase`) as more realistic for a library search box.
Initial demo used a generic six-title array with two duplicated titles created by the AI.	**Rejected:** replaced with the trace array `[C, Java, DSA, Java, SQL]` plus an adjacent-duplicate array, per the manual trace specification.
AI was ready to write the implementation immediately after step 2.	**Deferred:** switched to build mode only after the manual-trace requirements were finalized.

5. My Verification

1. Explained the linear search algorithm back to the AI *before* any code was written; the AI corrected two misconceptions I then incorporated: (a) why `-1` is used instead of `0` as the sentinel, and (b) that the best case is O(1) while the overall bound is O(n).
2. Produced the manual trace *by hand, before running any code*, counting every comparison: findFirst("Java") = 2 comparisons (index 1, duplicate at 3 never inspected), findFirst("Python") = 5 comparisons (missing → -1), plus the first-position, last-position, and adjacent-duplicate cases.
3. Compiled (`javac LibraryFinder.java`) and ran (`java LibraryFinder`), then compared the program's *Comparisons made* values against my hand trace: **all 8 searches matched exactly** (1, 5, 2, 5, 5, 5, 1, 4).
4. Reviewed the generated code line by line against the approved pseudocode plan and confirm it contains the inspected-index printing, early-return logic, full-scan counting, null guards, and case-insensitive comparison as specified.
5. I can rebuild and explain this solution (sentinel value, early exit vs. full scan, best/worst-case complexity) without AI assistance.

Academic Honesty Statement

I attest that the AI assistance above was used within the instructor's policy, that I verified all AI output against my own manual trace as described in Section 5, and that I take full responsibility for any inaccuracy in the submitted work. Portions of this declaration itself were drafted with AI assistance for formatting.

Kane Paul A. Mari — Signature Date: October 23, 2026